

TASK 04 — Saga Pattern & Distributed Workflow

Step 1 — Understand Saga

The **Saga Pattern** solves this by dividing one large transaction into a sequence of smaller transactions.

1. Saga Pattern

A **Saga** is a sequence of **local transactions**.

Example:

Create Order



Reserve Inventory



Process Payment



Confirm Order

Each step is handled by a different microservice.

If one step fails, previously completed steps are **compensated**.

Example:

Create Order ✓

Reserve Inventory ✓

Process Payment ✗



Release Inventory



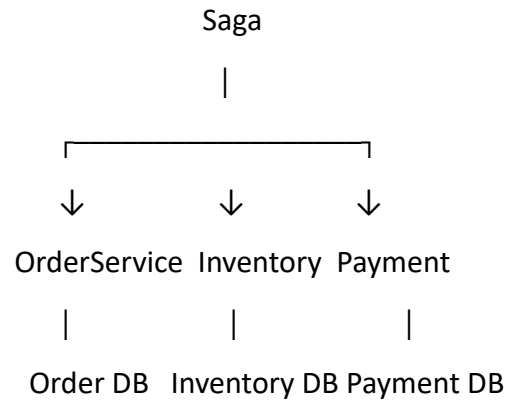
Cancel Order

So instead of a traditional rollback, Saga uses **compensating actions**.

2. Distributed Transaction

A **distributed transaction** is a transaction that involves multiple services or databases.

For example:



Each service performs its own transaction.

The entire business operation is considered successful only when all required steps complete successfully.

Example

An online shopping order:

1. Order Service creates order.
2. Inventory Service reserves product.
3. Payment Service processes payment.
4. Order Service confirms order.

3. Local Transaction

A **local transaction** happens inside one microservice and its own database.

For example:

Order Service



Create Order



Order Database



COMMIT

The Order Service does not directly control the Inventory or Payment database.

Similarly:

Inventory Service



Reserve Product



Inventory Database



COMMIT

Each local transaction should be completed independently.

4. Compensation

This is one of the **most important concepts in Saga**.

A compensation is an operation that **undoes the business effect** of an earlier successful transaction.

It is not always a database rollback.

Example

Create Order ✓

Reserve Inventory ✓

Process Payment ✗

We cannot simply roll back the payment service using one global database rollback.

Instead:

Payment failed



Release Inventory



Cancel Order

Original action	Compensation
-----------------	--------------

Create Order	Cancel Order
--------------	--------------

Reserve Inventory	Release Inventory
-------------------	-------------------

Charge Payment	Refund Payment
----------------	----------------

Simple definition

Compensation = business action used to undo a previously completed action.

5. Eventual Consistency

In a Saga, all services may not become consistent **immediately**.

Instead, they become consistent after all required operations and compensations finish.

This is called **eventual consistency**.

Example

Initially:

Order = CREATED

Inventory = RESERVED

Payment = PENDING

After payment succeeds:

Order = CONFIRMED

Inventory = RESERVED

Payment = COMPLETED

Or if payment fails:

Order = CANCELLED

Inventory = AVAILABLE

Payment = FAILED

The system may temporarily have intermediate states, but eventually it reaches a consistent business state.

Step 2 — Define the Workflow

Step 1: Create Order

The customer places an order.

Customer



Order Service



Create Order



Order DB

Example:

Order ID: 101

Product: Laptop

Status: CREATED

Step 2: Reserve Inventory

The Order Service requests the Inventory Service.

Order Service



Inventory Service



Reserve Laptop

If the laptop is available:

Inventory = RESERVED

If it is not available:

Inventory Reservation = FAILED

Then the Saga may cancel the order.

Step 3: Process Payment

After inventory is successfully reserved:

Inventory Reserved



Payment Service



Process Payment

If payment succeeds:

Payment = SUCCESS

If payment fails:

Payment = FAILED

Then compensation may be required:

Payment Failed



Release Inventory



Cancel Order

Step 4: Confirm Order

If everything succeeds:

Create Order ✓



Reserve Inventory ✓



Process Payment ✓



Confirm Order ✓

Final state:

Order Status = CONFIRMED

Payment = SUCCESS

Inventory = RESERVED

Complete Saga Flow

Successful transaction

CUSTOMER



Create Order



Failure scenario

Suppose payment fails:

Create Order

✓

↓

Reserve Inventory

✓

↓

Process Payment

X

↓

Payment Failed

↓

Release Inventory

↓

Cancel Order

This is the key idea of the **Saga Pattern**.

Simple Real-Life Example

Imagine ordering a phone online.

Normal case

1. Order phone



2. Reserve phone in warehouse



3. Pay ₹50,000



4. Confirm order



5. Ship phone

Payment failure

Order Created



Phone Reserved



Payment Failed



Release Phone



Cancel Order

The system does not leave the phone permanently reserved.

Saga Pattern:

A design pattern for managing distributed business transactions across multiple microservices.

Local Transaction:

A transaction performed within one service/database.

Distributed Transaction:

A business transaction involving multiple services/databases.

Compensation:

An operation that reverses the business effect of a previously successful step.

Eventual Consistency:

Services may temporarily have different states, but eventually reach a consistent business state.

Saga = Sequence of local transactions + compensation when a later step fails.

Step 3 — Define Failure Flow**1. Create Order**

First, the customer places an order.

Create Order ✓

The **Order Service** creates the order in its database.

Example:

Order ID: 101

Product: Laptop

Status: CREATED

2. Reserve Inventory

Next, the Inventory Service checks whether the product is available.

Reserve Inventory ✓

For example:

Laptop available = 10

↓

Reserve 1 laptop

↓

Available = 9

Reserved = 1

At this point, the inventory has been successfully reserved.

3. Payment Failed

Now the Payment Service tries to process the customer's payment.

Process Payment



FAILED X

Possible reasons could include:

- Insufficient balance
- Card declined
- Payment gateway unavailable
- Invalid payment details
- Network failure

The important point is that **payment failed after the previous steps had already succeeded.**

4. Release Inventory

Because payment failed, keeping the product reserved doesn't make sense.

So the Saga executes a **compensating transaction**:

Payment Failed



Release Inventory

The Inventory Service removes the reservation.

For example:

Before:

Available = 9

Reserved = 1

↓ Release Inventory

After:

Available = 10

Reserved = 0

This is called **compensation**.

Release Inventory is the compensation for Reserve Inventory.

5. Cancel Order

Finally, the Order Service cancels the order.

Release Inventory



Cancel Order

The order status becomes:

Order ID: 101

Status: CANCELLED

So the complete failure flow is:

Create Order



Reserve Inventory



Payment Failed



Release Inventory

↔ Compensation



Cancel Order

↔ Compensation

Step 4 — Define Service Responsibilities

Explains **which microservice is responsible for which data and operation**.

There are three main services in the image.

1. Order Service

Responsibility:

Order Service owns orders.

It manages everything related to orders.

For example:

Order Service



Order Database



Order ID

Customer

Product

Quantity

Price

Status

It can perform operations such as:

Create Order

Update Order

Confirm Order

Cancel Order

Get Order

Example:

POST /orders

creates an order.

2. Inventory Service

Responsibility:

Inventory Service owns inventory.

It manages product availability and reservations.

For example:

Inventory Service



Inventory Database



Product

Quantity

Available Stock

Reserved Stock

It can perform operations such as:

Check Stock

Reserve Inventory

Release Inventory

Update Stock

Example:

POST /inventory/reserve

reserves a product.

If payment fails:

POST /inventory/release

releases the reservation.

3. Payment Service

Responsibility:

Payment Service owns payments.

It handles payment-related operations.

For example:

Payment Service



Payment Database



Payment ID

Order ID

Amount

Payment Status

It can perform:

Process Payment

Check Payment

Refund Payment

Example:

POST /payments

processes a payment.

Step 5 — Create Events

services need a way to tell other services that something has happened.

1. OrderCreated

This event is generated when an order is successfully created.

Order Service



Create Order



OrderCreated

Example:

OrderCreated

```
{  
  "orderId": 101,  
  "productId": 501,  
  "quantity": 1  
}
```

The Inventory Service can listen for this event and reserve the product.

2. InventoryReserved

This event means that the requested product has been successfully reserved.

OrderCreated

↓

Inventory Service

↓

Reserve Inventory

↓

InventoryReserved

Example:

InventoryReserved

```
{  
  "orderId": 101,  
  "productId": 501,  
  "quantity": 1  
}
```

The Payment Service can then start processing the payment.

3. PaymentCompleted

This event is generated when payment succeeds.

InventoryReserved



Payment Service



Process Payment



PaymentCompleted

Example:

PaymentCompleted

```
{  
  "orderId": 101,  
  "amount": 50000,  
  "paymentId": 9001  
}
```

The Order Service can then confirm the order.

PaymentCompleted



Confirm Order

4. PaymentFailed

If payment is unsuccessful, the Payment Service publishes:

PaymentFailed

Example:

PaymentFailed

```
{
```



```
"orderId": 101,  
"reason": "Insufficient balance"  
}
```

Now the Saga needs to perform compensation.

PaymentFailed



Release Inventory



InventoryReleased



Cancel Order



OrderCancelled

5. InventoryReleased

This event means that previously reserved inventory has been released.

PaymentFailed



Inventory Service



Release Inventory



InventoryReleased

Example:

InventoryReleased

```
{  
  "orderId": 101,
```

```
"productId": 501,  
"quantity": 1  
}
```

This makes the product available for other customers.

6. OrderCancelled

After payment failure and inventory release, the Order Service cancels the order.

InventoryReleased



Order Service



Cancel Order



OrderCancelled

Example:

OrderCancelled

```
{  
  "orderId": 101,  
  "reason": "Payment failed"  
}
```

Complete Event Flow

Successful flow

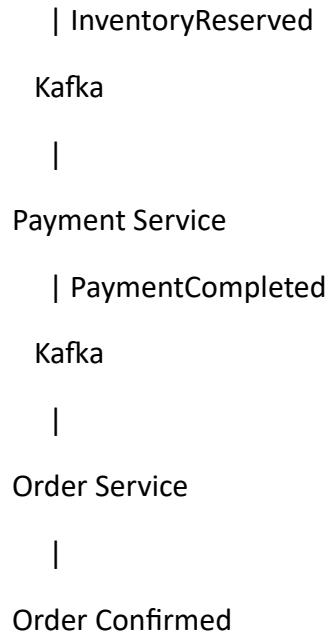
Order Service

| OrderCreated

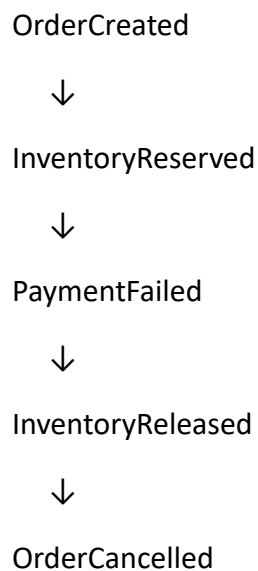
Kafka

|

Inventory Service



Failure flow

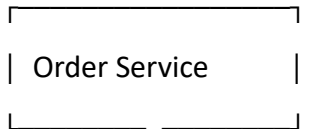


This is the **event-driven Saga flow**.

Step 6 — Implement Event Communication

This means instead of directly calling another service every time, services can communicate through **Apache Kafka**.

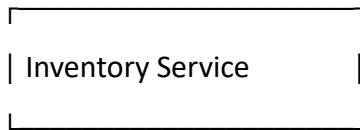
Basic architecture



| OrderCreated



| OrderCreated



What is Kafka doing here

Kafka acts as a **message/event broker** between microservices.

Instead of:

Order Service —————→ Inventory Service
direct call

we can use:

Order Service



Kafka



Inventory Service

The Order Service **publishes** an event.

The Inventory Service **consumes** that event.

Producer and Consumer

Two important Kafka concepts are:

Producer

The service that **sends/publishes** an event.

In this example:

Order Service



Producer



OrderCreated

Consumer

The service that **receives/consumes** an event.

OrderCreated



Kafka



Inventory Service



Consumer

So:

Order Service = Producer

Inventory Service = Consumer

Kafka Topic

Events are normally published to a Kafka **topic**.

For example:

Topic: order-events

The flow becomes:

Order Service



OrderCreated



order-events topic

↓

Inventory Service

Later, other events can use appropriate topics, for example:

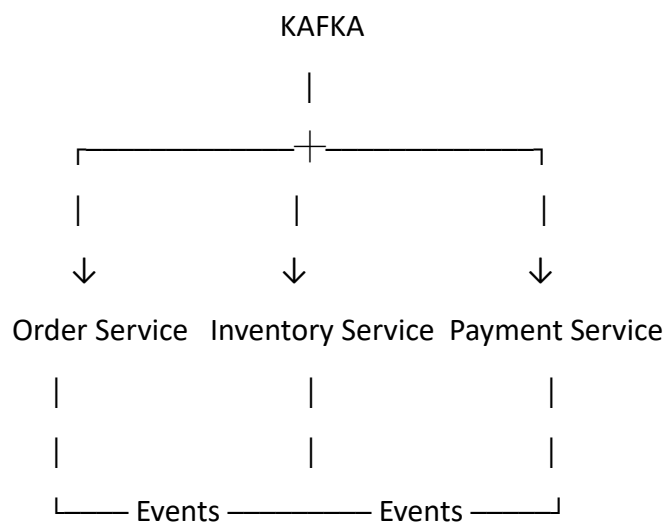
order-events

inventory-events

payment-events

Complete Saga with Kafka

A simplified architecture looks like this:



Successful transaction

1. Order Service

↓

OrderCreated

↓

Kafka

↓

2. Inventory Service

↓

InventoryReserved

↓

Kafka



3. Payment Service



PaymentCompleted



Kafka



4. Order Service



Order Confirmed

Failed transaction

OrderCreated



InventoryReserved



PaymentFailed



InventoryReleased



OrderCancelled

Why Kafka is useful for Saga

Kafka provides **asynchronous communication** between services.

For example, the Order Service doesn't necessarily have to wait for the Inventory Service through a direct synchronous HTTP call.

Instead:

Order Service



Publish Event



Kafka



Inventory Service

This provides:

- **Loose coupling** — services don't need to directly depend on each other's implementation.
- **Asynchronous communication** — services can process events independently.
- **Event history** — Kafka retains messages according to its topic retention configuration.
- **Scalability** — consumers can be scaled to process more events.
- **Failure handling** — consumers can retry processing messages depending on the application design.

Very Important Difference

Direct REST communication

Order Service

| HTTP

Inventory Service

Order Service directly calls Inventory Service.

Event-based communication

Order Service

| Event

Kafka

| Event

Inventory Service

Kafka sits between the services.

Step 7 — Implement Successful Workflow

1. Order

The customer places an order.

Customer



Order Service



Create Order

Example:

Order ID = 101

Status = CREATED

The Order Service stores the order in its own database.

2. Inventory

After the order is created, the Inventory Service reserves the required product.

OrderCreated



Inventory Service



Reserve Inventory

Example:

Product = Laptop

Quantity = 1

Status = RESERVED

The product is now held for this particular order.

3. Payment

After inventory is successfully reserved, the Payment Service processes the payment.

InventoryReserved



Payment Service



Process Payment

If the payment succeeds:

Payment = COMPLETED

4. Confirmed

Once payment is successful, the Order Service receives the success event and confirms the order.

PaymentCompleted



Order Service



Order = CONFIRMED

Final successful flow

Order Created



Inventory Reserved



Payment Completed



Order Confirmed

So the final state is:

Order → CONFIRMED

Inventory → RESERVED

Payment → COMPLETED

Step 8 — Implement Payment Failure

1. Order

First, the customer creates an order.

Order



CREATED

For example:

Order ID: 101

Status: CREATED

2. Inventory Reserved

The Inventory Service reserves the product.

OrderCreated



Inventory Service



InventoryReserved

Example:

Laptop:

Available = 9

Reserved = 1

The order is waiting for payment.

3. Payment Failed

The Payment Service tries to process the payment, but it fails.

InventoryReserved



Payment Service



Payment Failed

For example:

Payment Status = FAILED

Reason = Insufficient balance

At this point, the Saga cannot continue with order confirmation.

4. Inventory Released

Because the payment failed, we don't want to keep the product reserved.

Therefore, the Saga performs a **compensating transaction**.

Payment Failed



Release Inventory

Before release:

Available = 9

Reserved = 1

After release:

Available = 10

Reserved = 0

An event can be published: InventoryReleased

5. Order Cancelled

After the inventory is released, the Order Service cancels the order.

InventoryReleased



Order Service



OrderCancelled

Final state:

Order → CANCELLED

Inventory → AVAILABLE

Payment → FAILED

Successful Workflow Payment Failure Workflow

Order Created	Order Created
Inventory Reserved	Inventory Reserved
Payment Completed	Payment Failed
Order Confirmed	Inventory Released
—	Order Cancelled

Successful

Order



Inventory Reserved



Payment Completed



Confirmed

Failed

Order



Inventory Reserved



Payment Failed



Inventory Released



Order Cancelled

Step 9 — Add Status Tracking

1. PENDING

This is the **initial state**.

The customer has created an order, but the Saga has not completed the next steps yet.

Customer



Create Order



PENDING

Example:

Order ID: 101

Status: PENDING

2. INVENTORY_RESERVED

The Inventory Service successfully reserves the product.

PENDING



Reserve Inventory



INVENTORY_RESERVED

Example:

Order ID: 101

Status: INVENTORY_RESERVED

This means:

The product is available and has been reserved for this order.

3. PAYMENT_PROCESSING

Now the Payment Service is processing the payment.

INVENTORY_RESERVED



Start Payment



PAYMENT_PROCESSING

Example:

Order ID: 101

Status: PAYMENT_PROCESSING

This means the payment has not yet been completed or failed.

4. CONFIRMED

If payment succeeds, the order becomes confirmed.

PAYMENT_PROCESSING



Payment Successful



CONFIRMED

Final successful state:

Order Status = CONFIRMED

This means:

- Order was created
- Inventory was reserved
- Payment was successful
- Order is confirmed

Failure Statuses

5. PAYMENT_FAILED

If the payment fails:

PAYMENT_PROCESSING



Payment Failed



PAYMENT_FAILED

For example:

Order ID: 101

Status: PAYMENT_FAILED

Possible reasons:

- Insufficient balance
- Card declined
- Payment gateway error
- Invalid payment information

The Saga then starts compensation.

PAYMENT_FAILED



Release Inventory



CANCELLED

6. CANCELLED

The order becomes cancelled after the failure and compensation process.

PAYMENT_FAILED



Inventory Released



CANCELLED

Final state:

Order Status = CANCELLED

Complete Status Flow

Successful transaction

PENDING



INVENTORY_RESERVED



PAYMENT_PROCESSING



CONFIRMED

Payment failure

PENDING



INVENTORY_RESERVED



PAYMENT_PROCESSING



PAYMENT_FAILED



CANCELLED

The important point is that **the status tells us exactly what is happening with the order.**

Why Status Tracking Is Important

Imagine a customer asks:

"What is happening with my order?"

Without status tracking, it is difficult to know.

With status tracking:

Order Status:

PAYMENT_PROCESSING

we immediately know that inventory has already been reserved and payment is currently being processed.

It also helps developers and support teams troubleshoot failures.

For example:

Order 101

Status = PAYMENT_FAILED

We know that the failure occurred during the payment stage.

Example Database Table

The Order Service could store something like:

order_id	customer	amount	status
101	Customer A	₹50,000	CONFIRMED
102	Customer B	₹20,000	PAYMENT_PROCESSING
103	Customer C	₹15,000	PAYMENT_FAILED
104	Customer D	₹8,000	CANCELLED

A simple Java enum could represent these states:

```
public enum OrderStatus {  
    PENDING,  
    INVENTORY_RESERVED,  
    PAYMENT_PROCESSING,  
    CONFIRMED,  
    PAYMENT_FAILED,  
    CANCELLED  
}
```

Then the Order entity can have:

```
@Enumerated(EnumType.STRING)  
private OrderStatus status;
```

Step 10 — Test Failure Scenarios

1. Inventory Unavailable

Suppose the customer orders a laptop, but there are no laptops available.

PENDING



Check Inventory



Inventory Unavailable

The reservation fails.

The order should not continue to payment.

Possible result:

PENDING



Inventory Reservation Failed



CANCELLED

Example

Available Stock = 0

Customer requests = 1

Result: Inventory Unavailable

2. Payment Failure

Inventory is successfully reserved, but payment fails.

PENDING



INVENTORY_RESERVED



PAYMENT_PROCESSING



PAYMENT_FAILED

Now compensation is required:

PAYMENT_FAILED



Release Inventory



CANCELLED

3. Service Unavailable

Suppose the Payment Service is temporarily unavailable.

Order Service



Inventory Service



Payment Service



Service Unavailable

For example:

HTTP 503 Service Unavailable

The system should not immediately assume that the payment permanently failed.

Depending on the design, it may:

Payment Service unavailable



Retry



Payment Service available



Process Payment

Or, after retry attempts are exhausted, it may execute an appropriate failure/compensation flow.

4. Duplicate Event

In event-driven systems such as Kafka, an event may sometimes be delivered more than once.

For example:

OrderCreated



Kafka



Inventory Service

OrderCreated



Kafka



Inventory Service

The Inventory Service might receive the same event twice.

Problem

Without protection, it could reserve the same inventory twice.

For example:

First event → Reserve 1 laptop

Second event → Reserve another laptop

That could produce incorrect inventory.

Solution: Idempotency

The service should process the same event safely more than once.

For example:

Event ID = EVT-1001

First time:

EVT-1001 → Process ✓

Second time:

EVT-1001 → Already processed → Ignore

This is called **idempotent processing**.

5. Retry Scenario

Sometimes an operation fails temporarily.

For example:

Payment Service

↓

Temporary network error

↓

Retry

↓

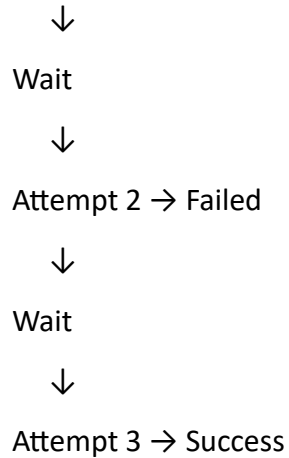
Payment Service

↓

Success

A common approach is:

Attempt 1 → Failed



The exact number of retries and delay should be configured according to the application.

A common strategy is **exponential backoff**, where the waiting time increases between attempts.

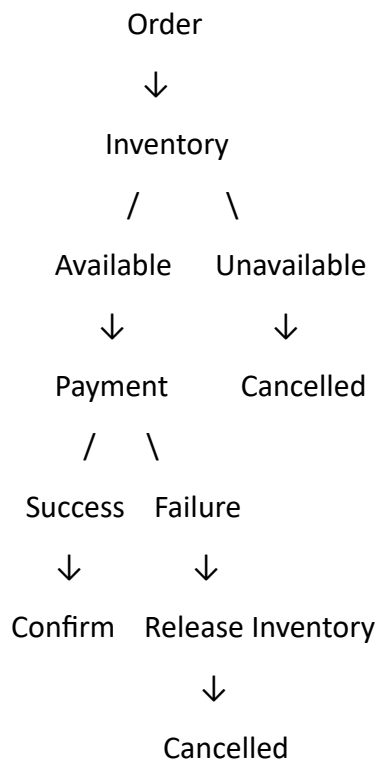
Example:

1st retry → wait 1 second

2nd retry → wait 2 seconds

3rd retry → wait 4 seconds

Complete Testing Picture :You should test your Saga with both **success and failure scenarios**.



And additionally test:

Service unavailable



Retry



Success / Failure

and:

Duplicate event



Idempotency check



Process only once

Step 11 — Create Saga Diagram

1. Success Flow

The image shows:

Order → Inventory → Payment → Confirm

This means all steps are completed successfully.

Step-by-step

Step 1 — Order

Order Created

The Order Service creates the order.



Step 2 — Inventory

Inventory Reserved

The Inventory Service reserves the product.



Step 3 — Payment

Payment Completed

The Payment Service successfully processes the payment.

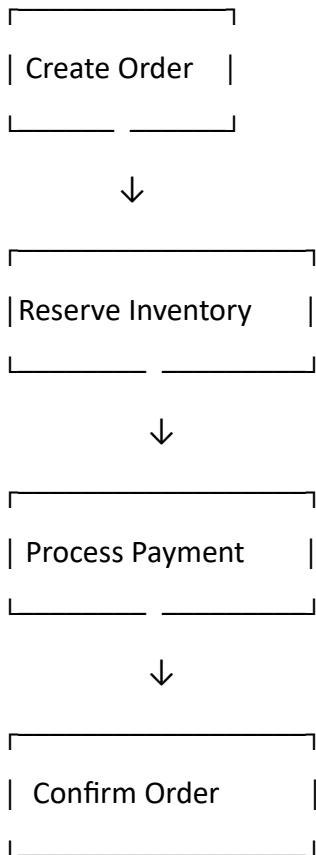


Step 4 — Confirm

Order Confirmed

The Order Service changes the order status to CONFIRMED.

Complete success flow



2. Failure Flow

The image shows:

Order → Inventory → Payment X



Compensation



Release Inventory



Cancel Order

Here, **payment fails** after the order and inventory operations have already succeeded.

Step 1 — Order

Order Created ✓

The order is created successfully.

Step 2 — Inventory

Inventory Reserved ✓

The product is successfully reserved.

For example:

Available stock = 10



Reserve 1



Available stock = 9

Step 3 — Payment Fails

The Payment Service tries to process the payment.

Payment X

For example:

Payment Failed

Reason: Insufficient balance

Now the Saga cannot continue to Confirm Order.

Instead, it goes into the **compensation process**.

3. Compensation

This is the most important part of the failure flow.

Because inventory was already reserved, the system needs to undo that business operation.

Payment Failed



Compensation

The compensation action is:

Release Inventory

So:

Reserved = 1



Release



Reserved = 0

Available = 10

4. Cancel Order

After releasing the inventory, the order is cancelled.

Release Inventory



Cancel Order

Final state:

Order → CANCELLED

Inventory → RELEASED

Payment → FAILED

So the complete failure flow is:

Create Order



Reserve Inventory



Payment Failed X



Compensation



Release Inventory



Cancel Order

Success vs Failure

Success

Order created

Inventory reserved

Payment completed

Order confirmed

Transaction completed

Failure

Order created

Inventory reserved

Payment failed

Release inventory

Cancel order